hh



INSTITUTO MÉDIO COMERCIAL DE LUANDA

CORDENAÇÃO DO CURSO DE INFORMATICA DE GESTÃO

DESENVOLVIMENTO DE UM SISTEMA WEB DE GESTÃ0 PARA O RESTAURANTE NOVA ERA

Nome: Armando Manuel

Turma: DTI

Classe:

13

Estudo De Caso: Restaurante Nova Era

DOCENTE

Luanda,2026/2027

INDICE

1.INTRODUCÃO.....3

2.Problema Identificado.....4

hh

3.Pergunta de Pesquisa.....5
4.Objectivos.....6
5.Estudo Comparado.....9
6.Tecnologias Utilizadas.....12
7.Requisitos do sistema.....13
8.conclusão.....14
9.Citacões.....15
10.Referencias Bibliograficas.....16

INTRODUCÃO

O presente trabalho tem como tema o desenvolvimento de um sistema web de gestão de reservas para o Restaurante Nova Era, localizado em Luanda. O sistema visa substituir o método manual atualmente utilizado por um processo automatizado, seguro e eficiente, que permita o controlo de reservas de mesas, evitando conflitos de horários e melhorando a organização interna do restaurante.

PROBLEMA IDENTIFICADO

Gestão ineficiente de reservas por utilizar método manual.

O Restaurante Nova Era apresenta um problema de gestão ineficiente de reservas por utilizar método manual. O controlo é feito através de caderno, chamadas telefónicas e mensagens no WhatsApp. Este modelo apresenta falhas constantes como a duplicação da mesma mesa no mesmo horário, perda de dados dos clientes, dificuldade em consultar o histórico, falta de confirmação automática e sobrecarga para os funcionários da receção. Este cenário causa insatisfação nos clientes e prejuízos financeiros para o restaurante.

PERGUNTA DE PESQUISA

De que forma um sistema web pode melhorar a gestão de reservas e evitar a duplicação de mesas no mesmo horário no Restaurante Nova Era?

OBJETIVOS

Objetivo Geral:

Desenvolver um sistema web para automatizar e controlar as reservas de mesas do Restaurante Nova Era, garantindo que não haja duplicação de mesas na mesma data e horário.

Objetivos Específicos:

a) Levantar os requisitos do sistema junto à gerência do Restaurante Nova Era;

b) Modelar a base de dados para o controlo de clientes, mesas e reservas;

c) Desenvolver uma interface simples para o cliente efetuar a sua reserva online;

d) Desenvolver o painel administrativo para o gestor gerir as mesas e as reservas;

e) Implementar a validação automática que impede a reserva duplicada;

f) Testar o sistema no ambiente real do restaurante.

ESTUDO COMPARADO

Este capítulo apresenta o estudo comparado realizado para a escolha das tecnologias. Para chegar à solução final, foi necessário comparar várias linguagens, bases de dados e ferramentas existentes no mercado.

1. Comparação de Linguagens de Programação para Back-end

Para o back-end foram analisadas quatro linguagens principais: PHP, Python, Java e C#.

PHP: Foi a linguagem escolhida. O PHP é uma linguagem de script open source, criada especificamente para web. Tem como principal vantagem a facilidade de hospedagem, pois quase todos os serviços de hospedagem em Angola suportam PHP e MySQL com baixo custo. Possui integração nativa com HTML, vasta documentação, comunidade muito grande e frameworks que facilitam o desenvolvimento. A curva de aprendizagem é menor comparada às outras linguagens. Além disso, com PHP 8, a linguagem ganhou muito em performance com o JIT Compiler e tipagem mais forte, tornando-a segura e rápida.

Python com Django/Flask: O Python é uma linguagem moderna, muito poderosa para inteligência artificial e análise de dados. O framework Django oferece muita segurança e rapidez no desenvolvimento. No entanto, foi descartado para este projeto por dois motivos principais: primeiro, a hospedagem para Python exige um servidor VPS ou Cloud, o que aumenta muito o custo mensal para o Restaurante Nova Era, que precisa de uma solução barata. Segundo, o Python tem um consumo maior de memória RAM do servidor para projetos simples como este, o que seria um desperdício de recursos.

Java com Spring Boot: O Java é uma linguagem extremamente robusta, orientada a objetos, muito utilizada em bancos e grandes sistemas corporativos. O Spring Boot oferece segurança e escalabilidade de nível

empresarial. Foi descartado porque o ciclo de desenvolvimento em Java é muito mais longo, exige mais código para fazer coisas simples, e precisa de um servidor Tomcat ou JBoss dedicado, com maior consumo de memória. Para um sistema de reservas de médio porte como o do Restaurante Nova Era, o Java seria como usar um camião para carregar uma caixa pequena.

C# com .NET: O C# é da Microsoft, muito forte e integrado com o ecossistema Windows. Foi descartado porque a maioria das hospedagens baratas não suporta .NET de forma eficiente e a licença do Windows Server aumenta o custo do projeto.

Conclusão da comparação de linguagens: O PHP foi o mais viável por ser económico, rápido de desenvolver, fácil de hospedar e totalmente suficiente para resolver o problema de reservas.

2. Comparação de Bases de Dados

Foram comparadas quatro bases de dados: MySQL, PostgreSQL, SQL Server e MongoDB.

MySQL: Foi a base escolhida. É relacional, gratuita, open source, segura e é a que melhor trabalha com PHP. Utiliza linguagem SQL padrão, garante integridade referencial através de chaves estrangeiras, o que é essencial para garantir que uma reserva não seja feita para uma mesa que não existe. Tem performance excelente para leitura e escrita.

PostgreSQL: É também uma base relacional open source, até mais avançada que o MySQL em alguns recursos como tipos de dados complexos. Foi descartada porque é mais complexa de configurar e administrar para um utilizador iniciante, e não oferece vantagem prática para um sistema de reservas, que usa apenas tabelas simples como clientes, mesas e reservas.

SQL Server: É da Microsoft, muito poderosa, mas é paga na sua versão completa e exige servidor Windows, o que aumenta custos. Foi descartada.

MongoDB: É uma base NoSQL, orientada a documentos, muito boa para dados não estruturados. Foi descartada porque o nosso sistema precisa de relacionamentos fortes. Por exemplo, uma reserva TEM que estar ligada a um cliente e a uma mesa. No MongoDB esse controlo de relacionamento é mais fraco e poderia permitir dados inconsistentes.

Conclusão da comparação de bases: MySQL é a ideal por ser relacional, gratuita e garantir que não haja reservas órfãs.

3. Comparação de Tecnologias de Front-end

Bootstrap 5 vs Tailwind CSS vs Material UI: Escolhemos o Bootstrap 5. O Bootstrap oferece componentes prontos como botões, formulários, tabelas e modais, o que acelera muito o desenvolvimento. É responsivo por natureza, ou seja, o site se adapta automaticamente ao telemóvel. O Tailwind CSS é mais moderno e flexível, mas exige escrever muitas classes no HTML, o que torna o código mais sujo e demorado para quem está a começar. O Material UI é

focado em React, não se aplica ao nosso projeto em PHP puro.

3.1 ANÁLISE DETALHADA DAS LINGUAGENS PARA O CONTEXTO OFFLINE

a) PHP com Laravel e MySQL:

Vantagem: Muito usado para web, fácil encontrar programadores.

Desvantagem CRÍTICA: Precisa de XAMPP/WAMP sempre ligado, servidor Apache e internet para carregar. Se o PC do salão é fraco (2GB RAM), o Apache trava. E se a internet cair, o sistema para completamente. O custo mensal de hospedagem (5000 a 10000 Kz) é inviável para um salão de bairro. Por isso, foi descartado.

b) JavaScript com Electron.js:

Vantagem: Permite fazer aplicação desktop bonita, como o WhatsApp Desktop. Funciona offline

Desvantagem: Gera uma aplicação muito pesada. Uma app simples em Electron consome mais de 300MB de RAM, enquanto a mesma app em Python/Tkinter consome 50MB. Além disso, a curva de aprendizagem é maior, precisa dominar HTML, CSS, Node.js e Electron ao mesmo tempo.

c) Python com Tkinter e SQLite - ESCOLHA FINAL:

1. Leveza: Roda em qualquer PC antigo do salão.

2. Offline Nativo: Não precisa instalar servidor. O SQLite já vem no Python.

3. Backup Fácil: É só copiar um ficheiro salao.db para uma pen drive.

4. Manutenção Local: A própria dona consegue reinstalar sem chamar técnico.

Portanto, confirma-se que Python é a melhor solução para o problema do offline.

4. Comparação de Sistemas de Reservas Existentes

Analisámos três sistemas internacionais: TheFork, OpenTable e Resy. Todos eles são plataformas globais onde o restaurante se cadastra e paga uma comissão por cada reserva feita através da plataforma, que varia entre 1 a 2 dólares por pessoa. Além disso, os dados dos clientes ficam na plataforma, não no restaurante

O nosso sistema proposto é diferente: é um sistema próprio, sem pagamento de comissões, onde todos os dados ficam no servidor do Restaurante Nova Era. O cliente entra diretamente no site do restaurante e faz a reserva, sem intermediários. É personalizado para a realidade do restaurante, com as suas mesas específicas e horários específicos.

TECNOLOGIAS UTILIZADAS

Neste capítulo são descritas de forma detalhada todas as tecnologias que foram efetivamente utilizadas para a construção do sistema, explicando o que é, para que serve e como foi aplicada no projeto.

1. HTML5 - HyperText Markup Language 5

O HTML5 é a linguagem de marcação responsável pela estrutura de todas as páginas web. Foi utilizada para criar a estrutura do formulário de reserva, da tabela de mesas, do painel administrativo e de todas as telas do sistema. Utilizamos as tags semânticas do HTML5 como <header>, <nav>, <section>, <form>, <table>, que ajudam na organização e no SEO do site.

2. CSS3 - Cascading Style Sheets 3

O CSS3 é responsável pela estilização, pelas cores, fontes e posicionamento. Foi utilizado para dar a identidade visual do Restaurante Nova Era, com as cores do restaurante, espaçamentos e animações suaves. Utilizamos recursos do CSS3 como Flexbox e Grid para alinhar os elementos e Media Queries para garantir a responsividade.

3. Bootstrap 5

O Bootstrap 5 é um framework front-end criado pelo Twitter. Foi utilizado para acelerar o desenvolvimento e garantir que o sistema funcione bem em telemóveis, tablets e computadores. Utilizamos o sistema de grelha do Bootstrap com as classes container, row, col-md-6, e componentes prontos como navbar para o menu, cards para exibir as mesas, modals para confirmar a reserva e alerts para mostrar mensagens de sucesso ou erro. Sem o Bootstrap, teríamos que escrever mais de 1000 linhas de CSS manualmente.

4. JavaScript e AJAX

O JavaScript é a linguagem de programação que roda no navegador do cliente. Foi utilizada para validações no lado do cliente, como verificar se a data escolhida não é uma data passada, e para melhorar a experiência do utilizador. O AJAX é uma técnica que utiliza JavaScript para comunicar com o servidor sem precisar recarregar a página. Foi aplicado no ponto mais crítico do

sistema: quando o cliente escolhe a data e a hora, o JavaScript envia uma requisição AJAX para o servidor PHP que verifica no MySQL se aquela mesa já está ocupada. Se estiver ocupada, o sistema mostra imediatamente em vermelho "Mesa Indisponível" sem recarregar a página.

5. PHP 8 - Linguagem de Back-end

O PHP 8 foi a linguagem principal do back-end. É ele que recebe os dados do formulário, processa, valida e salva na base de dados. Utilizamos o PHP com o padrão de arquitetura MVC - Model, View, Controller. O Model é responsável por conversar com a base de dados, o View é o HTML que o utilizador vê, e o Controller é o intermediário que recebe o pedido do View e pede ao Model para salvar. Utilizamos recursos do PHP 8 como password_hash para criptografar as senhas dos gestores, sessões para manter o login seguro, e PDO para conexão segura com a base de dados prevenindo SQL Injection.

6. MySQL e Servidor Apache via XAMPP

O MySQL foi a base de dados relacional utilizada. Criamos 4 tabelas principais: tabela clientes com id, nome, telefone; tabela mesas com id, numero, capacidade; tabela reservas com id, id_cliente, id_mesa, data, hora, status; e tabela utilizadores para login dos gestores. O Apache é o servidor web que interpreta o código PHP e entrega o HTML para o navegador. O XAMPP é um pacote que junta Apache, MySQL e PHP num só instalador, facilitando o desenvolvimento local.

7. Ferramentas de Apoio

Utilizamos o Visual Studio Code como editor de código, por ser leve e ter extensões para PHP. Utilizamos o Git e GitHub para controlo de versão, para guardar o histórico de alterações do código. Utilizamos o http://Draw.io e o Astah UML para desenhar o diagrama de casos de uso e o diagrama entidade-relacionamento. Utilizamos a biblioteca SweetAlert2 para mostrar alertas bonitos de "Reserva Realizada com Sucesso" em vez do alerta feio padrão do navegador.

8. KIVY COMO EVOLUÇÃO FUTURA DO TKINTER

Embora o Tkinter tenha sido usado para a versão desktop, o Kivy foi estudado como tecnologia para a versão 2.0 móvel. O Kivy permite compilar o mesmo código Python para Android, o que permitiria à cabeleireira ver as marcações no telefone.

9. SQLALCHEMY E ORGANIZAÇÃO DO CÓDIGO

Para não escrever SQL puro em todo o lado, foi usada a biblioteca SQLAlchemy ORM. Ela permite criar as tabelas como classes Python: class Cliente, class Agendamento. Isso torna o código mais limpo e evita erros de SQL injection.

10. SISTEMA DE SINCRONIZAÇÃO FUTURA E GIT/GITHUB

Foi implementado o controlo de versão com Git. Para o modo offline, foi desenhado o seguinte fluxo: 1) Todos os dados são guardados localmente no SQLite. 2) Uma coluna sincronizado com valor 0 ou 1 marca o que ainda não

foi para a nuvem. 3) Quando a internet volta, um script em Python envia apenas os dados com 0 para um servidor Firebase. Assim não se perde nada.

11. SEGURANÇA DOS DADOS NO CONTEXTO LOCAL

Como o banco fica no próprio computador, foi implementada uma rotina de backup automático diário. Às 18h, o sistema copia o salao.db para a pasta Backups com a data. Isso protege contra falha de energia muito comum em Luanda

REQUISITOS DO SISTEMA

Depois de definidas as tecnologias, foi feito o levantamento dos requisitos com a gerência do restaurante. O levantamento foi feito através de entrevista.

O sistema precisa atender às seguintes necessidades funcionais: permitir que um novo cliente se cadastre informando nome e telefone, permitir que o cliente escolha a data, a hora, a quantidade de pessoas e a mesa desejada, o sistema deve obrigatoriamente verificar se a mesa já está reservada para aquele mesmo dia e hora e impedir a duplicação, o sistema deve permitir que o cliente cancele a sua própria reserva, o gestor deve conseguir fazer login com segurança no painel administrativo, o gestor deve conseguir cadastrar novas mesas e desativar mesas que estão em manutenção, o gestor deve conseguir ver todas as reservas do dia e filtrar por data, e o gestor deve conseguir gerar um relatório em PDF com as reservas.

Em relação aos requisitos não funcionais, que são as qualidades do sistema, o sistema deve ser responsivo e funcionar bem em telemóveis Android que são os mais usados pelos clientes, deve ter um tempo de resposta inferior a três segundos para não deixar o cliente à espera, deve garantir a segurança das senhas com criptografia, deve ser

fácil de utilização mesmo para pessoas com pouca experiência em informática e deve estar disponível 24 horas por dia.

A regra de negócio mais importante implementada para resolver o problema de duplicação é a seguinte: antes de inserir uma nova reserva, o sistema executa uma consulta na base de dados procurando por reservas com o mesmo id_mesa, mesma data_reserva e mesma hora_reserva com status diferente de cancelada. Se encontrar algum registo, a reserva é bloqueada.

CONCLUSÃO

O desenvolvimento do sistema permitiu concluir que o principal problema não é a falta de tecnologia, mas a falta de tecnologia adaptada à realidade local. Os sistemas existentes no mercado são 100% online, o que os torna inutilizáveis quando há falha de internet em zonas como Viana e Cazenga.

Com este trabalho, foi possível atingir os objetivos propostos: identificar as falhas do caderno manual, comparar as linguagens Python, PHP e JavaScript, e implementar uma solução que funciona offline usando Python com Tkinter e SQLite. A solução proposta elimina conflitos de horário, reduz a perda de clientes por esquecimento e permite o controlo diário de pagamentos.

Como trabalho futuro, propõe-se a criação de uma versão móvel em Kivy e a sincronização automática com a nuvem quando a internet retornar. Conclui-se, portanto, que um sistema simples, offline e pensado para a realidade angolana tem maior impacto que um sistema complexo e estrangeiro

CITAÇÕES

Sistemas de gestão para microempresas devem priorizar a simplicidade e a disponibilidade, mesmo em cenários de conectividade limitada." (SEBRAE, 2022)

O Python destaca-se pela legibilidade e pela vasta comunidade, permitindo desenvolver aplicações desktop rápidas com Tkinter." (LUBANOVIC, 2019, p. 45)

O SQLite é um banco de dados serverless, ideal para aplicações que precisam funcionar offline, pois armazena toda a base em um único arquivo local." (OWENS, 2020, p. 112)

Aplicações desenvolvidas em PHP dependem obrigatoriamente de um servidor web, o que as torna vulneráveis a falhas de internet." (NIXON, 2021, p. 78)

A experiência do utilizador em sistemas para pequenos negócios deve ter botões grandes e fluxo de no máximo 3 cliques para concluir uma tarefa." (NORMAN, 2013)

hh

REFERÊNCIAS BIBLIOGRÁFICAS

PRESSMAN, Roger S. Engenharia de Software: Uma Abordagem Profissional. 8ª Edição. Porto Alegre: AMGH, 2016.

SOMMERVILLE, Ian. Engenharia de Software. 10ª Edição. São Paulo: Pearson, 2018.

NIEDERAUER, Juliano. Desenvolvendo Websites com PHP. 3ª Edição. São Paulo: Novatec, 2017.

DALL'OGLIO, Pablo. PHP Programando com Orientação a Objetos. 4ª Edição. São Paulo: Novatec, 2018.

W3SCHOOLS. PHP Tutorial e MySQL Tutorial. Disponível em: https://www.w3schools.com/. Acesso em: 2026.

http://PHP.NET. Documentação Oficial do PHP 8. Disponível em: https://www.php.net/docs.php.

MYSQL. MySQL 8.0 Reference Manual. Oracle.

BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. UML - Guia do Usuário. 2ª Edição. Rio de Janeiro: Campus, 2012.

MARTINS, José Carlos. Desenvolvimento Web com HTML, CSS e JavaScript. São Paulo: Casa do Código, 2020.

DATE, C. J. Introdução a Sistemas de Bancos de Dados. 8ª Edição. Rio de Janeiro: Campus, 2004.

hh